Tree Data Structure

Dr. Nafis Sadeq
Assistant Professor
Department of Computer Science and Engineering
East West University

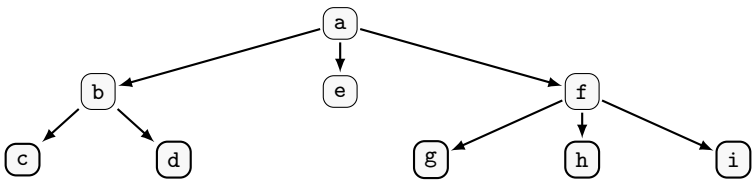


Outline

1. Tree Data Structure
2. Tree Terminology
3. Computing Height and Depth
4. Binary Tree
5. Tree Traversal

Introduction

- A **tree** is a hierarchical structure of nodes connected by edges.
- Captures "part-of" or "is-composed-of" relationships.
- Common shapes: *rooted* trees (single distinguished root), *binary* trees (degree ≤ 2), k-ary trees, ordered/unordered.



Applications

- File systems (directories/files)
- Expression trees (compilers)
- DOM trees (web browsers)
- Search trees (BST/AVL/Red-Black)
- Tries / prefix indexes
- Heaps (priority queues)
- Scene graphs (graphics)
- Game trees / decision trees

Formal Definition

A tree is a connected, acyclic graph.

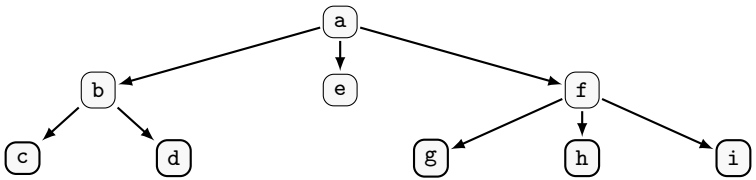
A **rooted tree** $T = (V, E, r)$ satisfies:

- $r \in V$ is the **root**. Every node except the root has exactly one **parent**.
- An edge implies a **parent** $\rightarrow$ **child** relation (directed away from the root).
- The **degree** of a node is its number of children; the tree's **arity** is $\max$ degree.
- If left/right order among children matters, the tree is **ordered**; otherwise **unordered**.

Tree Terminology

- **Root:** unique top node (depth 0).
- **Internal node:** node with ≥ 1 child.
- **External node (Leaf):** node with 0 children.
- **Ancestors/Descendants:** along parent/child chains.
- **Depth of v:** #edges from root to v (root depth $= 0$).
- **Height of v:** longest path (in edges) from v down to a leaf; tree height $=$ height of root.
- **Siblings:** nodes sharing the same parent.
- **Subtree at v:** v plus all its descendants.

Example Tree



- **root**: a **internal**: b, f **leaves**: c, d, e, g, h, i
- **siblings**: {b, e, f}, {c, d}, {g, h, i}
- **ancestors of d**: {a, b} **descendants of f**: {g, h, i}
- **depths**: depth(a)=0, depth(c)=2, depth(h)=2
- **heights**: height(c)=0, height(b)=1, height(a)=2

Finding the Height of a Binary Tree

```
struct Node {
  int val;
  Node* left;
  Node* right;
  Node(int v): val(v), left(nullptr), right(nullptr) {}
};

int height(Node* u){
  if(!u) return -1;
  int hl = height(u->left);
  int hr = height(u->right);
  return 1 + (hl > hr ? hl : hr);
}
```

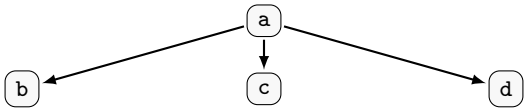
Finding the Depth of a Target Node from the Root

```cpp
struct Node {
  int val;
  Node* left;
  Node* right;
  Node(int v): val(v), left(nullptr), right(nullptr) {}
};

int depth_from(Node* u, int x){
  if(!u) return -1;
  if(u->val == x) return 0;
  int L = depth_from(u->left, x);
  if(L != -1) return 1 + L;
  int R = depth_from(u->right, x);
  if(R != -1) return 1 + R;
  return -1;
}
```

Returns -1 when x does not occur in the tree.

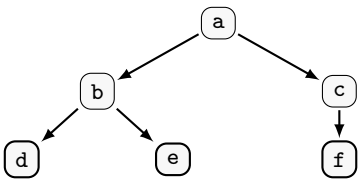
Ordered Trees



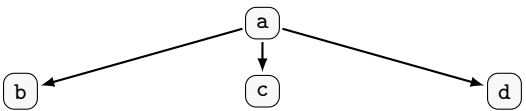
Children of a have a defined left-to-right order: `(b, c, d)`.

Binary Tree

In a binary tree, every node has at most two children (left/right).



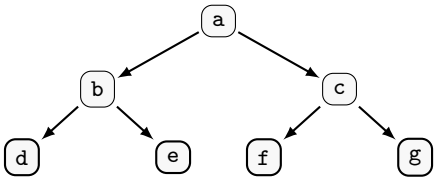
Binary Tree



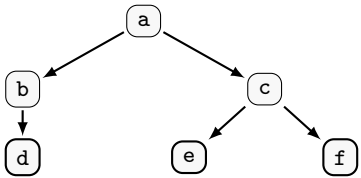
Not a binary tree

Proper Binary Tree

In a proper binary tree, every **internal** node has **exactly two** children.



Proper binary tree



Not a proper binary tree

Relationships in a Proper Binary Tree

- Let:
  - h = height of the tree (by edges)
  - n = total number of nodes
  - i = number of internal nodes
  - l = number of leaf (external) nodes
- For a **proper (full) binary tree:**

$$n = 2i + 1, \quad l = i + 1, \quad n = 2l - 1$$

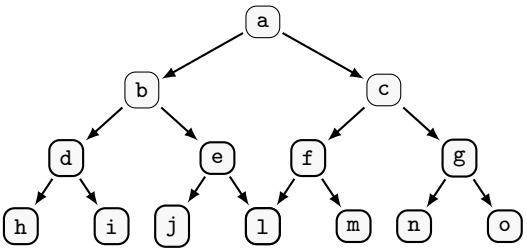
- The **maximum number of nodes** at height h:

$$n_{\text{max}} = 2^{h+1} - 1$$

- The **maximum number of leaves** at height h:

$$l_{\text{max}} = 2^h$$

Relationships in a Proper Binary Tree



- **Height:** $h = 3$
- **Internal nodes:** 7 (a,b,c,d,e,f,g)
- **Leaves:** 8 (h,i,j,k,l,m,n,o)
- **Total nodes:** $n = 15$
- **Relations:**

$$n = 2i + 1 = 15, \quad l = i + 1 = 8$$

Tree Traversal Types

- **Depth First Search (DFS)** explores one branch completely before backtracking.
  - **Preorder:** root, left, right
  - **Inorder:** left, root, right
  - **Postorder:** left, right, root
- **Breadth First Search (BFS)** explores level by level (**Level Order**).

Preorder Traversal

Order: root, left, right

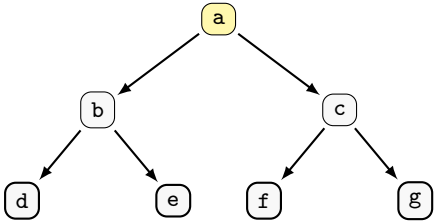
Steps:

1. Visit current node.
2. Recurse on left subtree.
3. Recurse on right subtree.

```
void preorder(Node* u){
  if(!u) return;
  visit(u);           // root
  preorder(u->left); // left
  preorder(u->right);// right
}
```

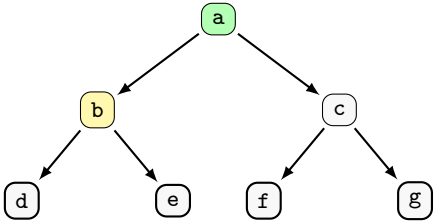
Preorder Traversal

unvisited current visited



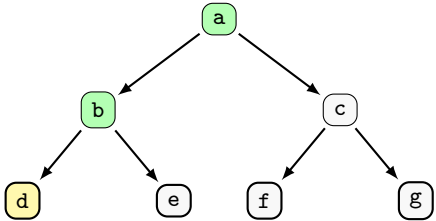
Preorder Traversal

unvisited current visited



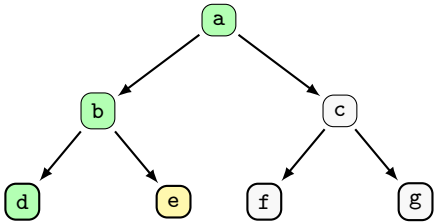
Preorder Traversal

unvisited current visited



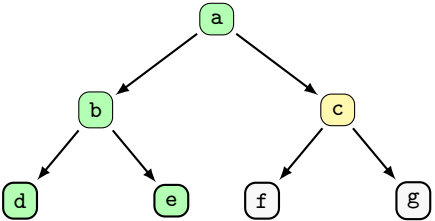
Preorder Traversal

unvisited current visited



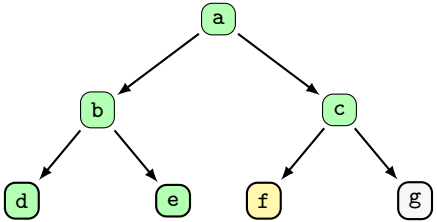
Preorder Traversal

unvisited current visited



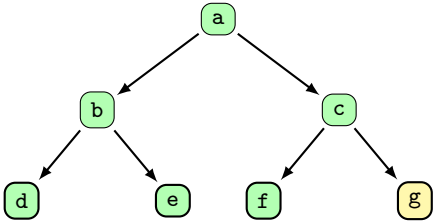
Preorder Traversal

unvisited current visited



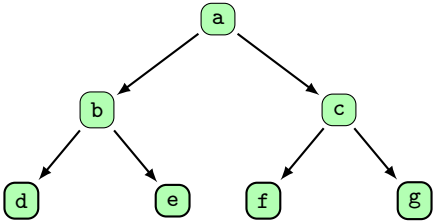
Preorder Traversal

unvisited current visited



Preorder Traversal

unvisited current visited



Visit order: a, b, d, e, c, f, g

Inorder Traversal

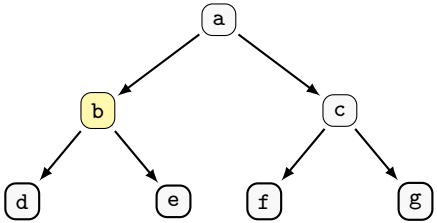
Order: left, root, right

1. Recurse on left subtree.
2. Visit current node.
3. Recurse on right subtree.

```
void inorder(Node* u){
  if(!u) return;
  inorder(u->left);  // left
  visit(u);          // root
  inorder(u->right); // right
}
```

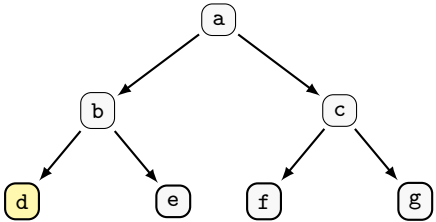
Inorder Traversal

unvisited current visited



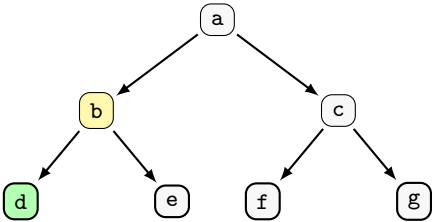
Inorder Traversal

unvisited current visited



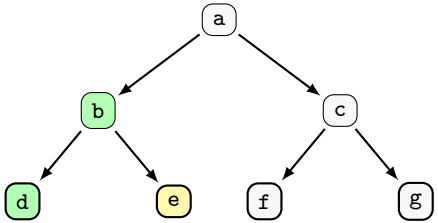
Inorder Traversal

unvisited current visited



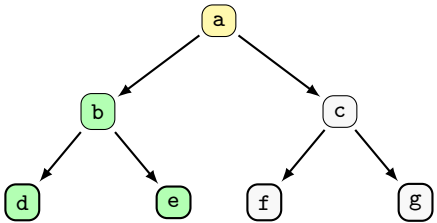
Inorder Traversal

unvisited current visited



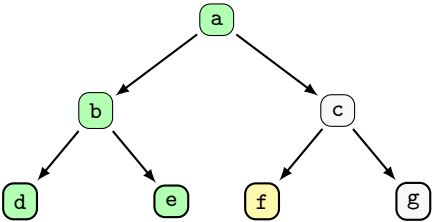
Inorder Traversal

unvisited current visited



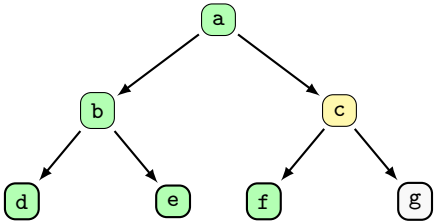
Inorder Traversal

unvisited current visited



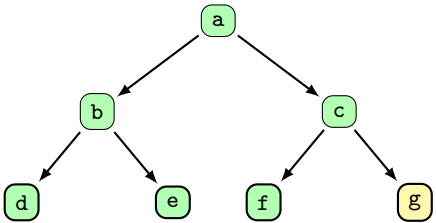
Inorder Traversal

unvisited current visited



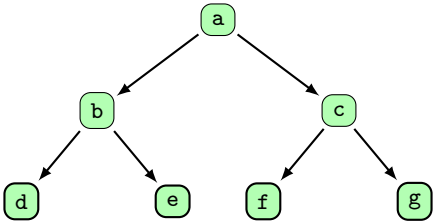
Inorder Traversal

unvisited current visited



Inorder Traversal

unvisited current visited



Visit order: d, b, e, a, f, c, g

Postorder Traversal

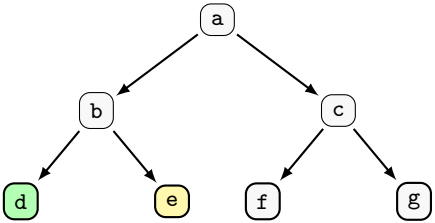
Order: left, right, root

1. Recurse on left subtree.
2. Recurse on right subtree.
3. Visit current node.

```
void postorder(Node* u){
  if(!u) return;
  postorder(u->left);  // left
  postorder(u->right); // right
  visit(u);            // root
}
```

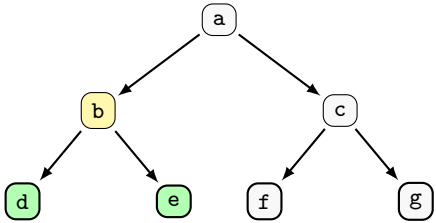
Postorder Traversal

unvisited current visited



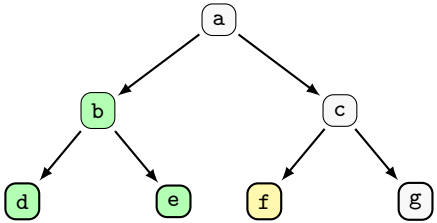
Postorder Traversal

unvisited current visited



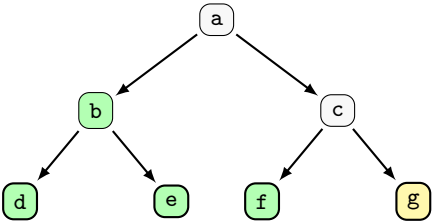
Postorder Traversal

unvisited current visited



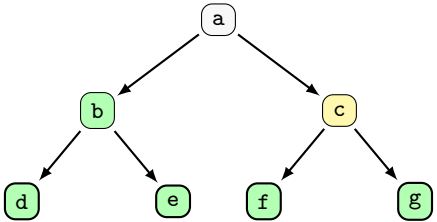
Postorder Traversal

unvisited current visited



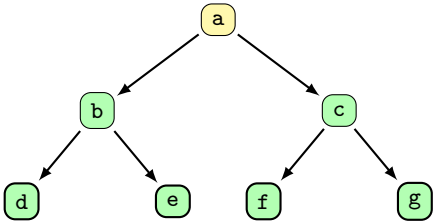
Postorder Traversal

unvisited current visited



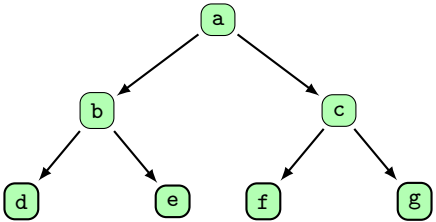
Postorder Traversal

unvisited current visited



Postorder Traversal

unvisited current visited



Visit order: d, e, b, f, g, c, a

Level Order Traversal

Order: level by level from root to leaves (left $\rightarrow$ right).

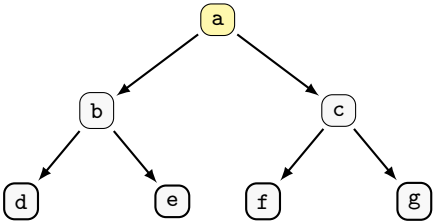
Steps:

1. Push root into a queue.
2. While queue not empty: pop front, visit it, enqueue its children.

```
#include <queue>
void levelorder(Node* root){
  if(!root) return;
  std::queue<Node*> q;
  q.push(root);
  while(!q.empty()){
    Node* u = q.front(); q.pop();
    visit(u);
    if(u->left)  q.push(u->left);
    if(u->right) q.push(u->right);
  }
}
```

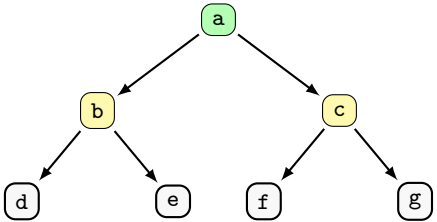
Level Order Traversal

unvisited current visited



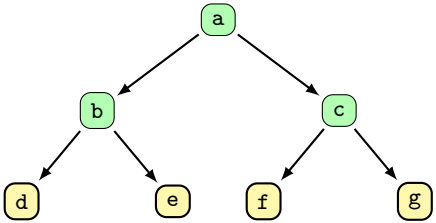
Level Order Traversal

unvisited current visited



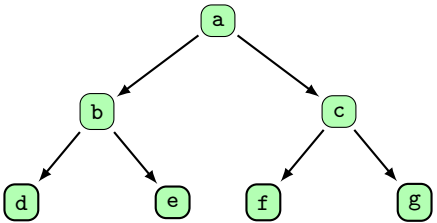
Level Order Traversal

unvisited current visited



Level Order Traversal

unvisited current visited



Visit order: a, b, c, d, e, f, g